

If you're preparing for a **real-time Data Engineer / PySpark project**, below is a complete PySpark ETL script from scratch with best practices.

Scenario

- Read employee data from CSV
- Clean data
- Remove duplicates
- Handle null values
- Add audit columns
- Write data to Parquet/Delta

1. Create Spark Session

</> Python



▶ Run

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Employee_ETL") \
    .getOrCreate()

spark.sparkContext.setLogLevel("ERROR")
```

2. Read Source File

</> Python



▶ Run

```
df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("/FileStore/data/employees.csv")

df.show()
```

Sample Data:

```
emp_id,name,email,salary,dept
101,Ram,ram@gmail.com,50000,IT
102,Sam,sam@gmail.com,60000,HR
103,John,john@gmail.com,70000,IT
103,John,john@gmail.com,70000,IT
```



3. Check Schema

</> Python



▶ Run

```
df.printSchema()
```

Output:

```
root
 |-- emp_id: integer
 |-- name: string
 |-- email: string
 |-- salary: integer
 |-- dept: string
```



4. Remove Duplicate Records

</> Python



▶ Run

```
df = df.dropDuplicates()
```

Or based on business key:

</> Python



▶ Run

```
df = df.dropDuplicates(["emp_id"])
```

5. Handle Null Values

Check null count:

</> Python



▶ Run

```
from pyspark.sql.functions import col, count, when

df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).show()
```

Replace nulls:

</> Python



▶ Run

```
df = df.fillna({
    "dept": "UNKNOWN",
    "salary": 0
})
```

6. Data Type Casting

</> Python



▶ Run

```
from pyspark.sql.functions import col

df = df.withColumn(
    "salary",
    col("salary").cast("double")
)
```

7. Filter Data

</> Python



▶ Run

```
it_df = df.filter(
    col("dept") == "IT"
)
```

8. Add Derived Columns

</> Python



▶ Run

```
from pyspark.sql.functions import lit

df = df.withColumn(
    "bonus",
    col("salary") * 0.10
)
```

9. Add Audit Columns

</> Python



▶ Run

```
from pyspark.sql.functions import current_timestamp

df = df.withColumn(
    "load_date",
    current_timestamp()
)
```

10. Sort Data

</> Python



▶ Run

```
df = df.orderBy(
    col("salary").desc()
)
```

11. Aggregation

Department-wise Salary

</> Python



▶ Run

```
from pyspark.sql.functions import sum

dept_salary = df.groupBy("dept") \
    .agg(
        sum("salary").alias("total_salary")
    )

dept_salary.show()
```

12. Window Function

Highest salary employee per department

</> Python



▶ Run

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

window_spec = Window.partitionBy("dept") \
```

```
        .orderBy(col("salary").desc())

df_rank = df.withColumn(
    "rn",
    row_number().over(window_spec)
)

top_emp = df_rank.filter(
    col("rn") == 1
)

top_emp.show()
```

13. Join Example

Department Table

</> Python



▶ Run

```
dept_df = spark.createDataFrame([
    (1, "IT"),
    (2, "HR"),
    (3, "FINANCE")
], ["dept_id", "dept"])
```

Join

</> Python



▶ Run

```
final_df = df.join(
    dept_df,
    "dept",
    "left"
)
```

14. Write Data to Parquet

</> Python



▶ Run

```
final_df.write \
    .mode("overwrite") \
    .parquet("/FileStore/output/employees")
```

15. Write Data to Delta

</> Python



▶ Run

```
final_df.write \
    .format("delta") \
    .mode("overwrite") \
    .save("/FileStore/delta/employees")
```

16. Read Delta Table

</> Python



▶ Run

```
delta_df = spark.read \
    .format("delta") \
    .load("/FileStore/delta/employees")

delta_df.show()
```

17. Incremental Load Pattern

</> Python



▶ Run

```
from pyspark.sql.functions import max

last_loaded_date = "2026-06-01"

incremental_df = df.filter(
    col("updated_date") > last_loaded_date
)
```

18. Logging

</> Python



▶ Run

```
record_count = df.count()

print(f"Source Record Count: {record_count}")
```

Real-Time ETL Flow

Source (Oracle/SQL Server/S3)

|

V

Read Data

|

Data Validation

|

Remove Duplicates

|

Null Handling

|

Transformations

|

Joins

|

Aggregations

|

Audit Columns

|

Write Delta/Parquet

|

Gold Layer Reporting



Oracle JDBC Read Example (Databricks)

<> Python



▶ Run

```
jdbc_url = "jdbc:oracle:thin:@//host:1521/service"

oracle_df = spark.read.format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "EMPLOYEES") \
    .option("user", "hr") \
    .option("password", "password") \
    .option("driver", "oracle.jdbc.OracleDriver") \
    .load()

oracle_df.show()
```

This is the type of end-to-end PySpark code commonly used in production ETL pipelines and Data Engineer interviews.